LAB 1

1. Cel: Prosty kalkulator własności funkcji liniowej, kwadratowej i wykładniczej oraz ich przebiegu na przedziale.
2. Przygotuj interfejs *Printable* z metodą *void print()*
3. Przygotuj klasę abstrakcyjną *Function* zawierającą abstrakcyjne metody:
    - double calculateValue(double x) – wartość funkcji w punkcie x
    - double[] calculateZeros() – miejsca zerowe; pusta tablica, gdy ich brak
4. Zaimplementuj klasy *LinearFunction*, *QuadraticFunction* i *ExponentialFunction*. Każda dziedziczy po *Function* i implementuje *Printable*:

Klasa	Wzór i parametry konstruktora	Niepoprawne dane (wyjątek)
LinearFunction	$f(x) = ax + b$	a = 0 (funkcja stała, nieskończenie wiele lub brak miejsc zerowych)
QuadraticFunction	$f(x) = ax^2 + bx + c$	a = 0 (to nie jest funkcja kwadratowa)
ExponentialFunction	$f(x) = a^x + b$	$a \leq 0$ lub a = 1

    - Do klas dopisać dowolne metody i pola pomocnicze, np. *calculateDelta()* i *calculateVertex()* w funkcji kwadratowej albo *isIncreasing()*
    - Każda klasa ma konstruktor ze współczynnikami typowymi dla danej funkcji
    - Metoda *print()* wypisuje wzór funkcji, współczynniki i miejsca zerowe, a dla kwadratowej także deltę i wierzchołek (p, q)
    - W przypadku podania niepoprawnych danych podczas konstrukcji obiektu – wyjątek powinien zostać rzucony
5. Zaimplementuj klasę `FunctionOnInterval`, czyli przebieg funkcji na przedziale. Obejmuje ona dowolną funkcję typu `Function` (użytkownik wybiera liniową, kwadratową lub wykładniczą) oraz pola: początek przedziału, koniec przedziału i krok. Oblicz:
    - tabelę wartości funkcji na przedziale z zadanym krokiem,
    - najmniejszą i największą wartość w tabeli,
    - przybliżone pole pod wykresem (całkę oznaczoną) metodą trapezów: suma po kolejnych odcinkach z $(f(x_i) + f(x_{i+1})) / 2 \cdot \text{krok}$.

    Wykorzystaj w tym celu abstrakcję. *FunctionOnInterval* korzysta wyłącznie z *calculateValue()*, bez *instanceof*. Klasa implementuje *Printable*, a konstruktor waliduje dane: funkcja ≠ null, początek < koniec, 0 < krok ≤ długość przedziału.

6. Stwórz prosty konsolowy interfejs użytkownika, oparty o `while` i `switch`, umożliwiający:
    - wybór funkcji;
    - podanie współczynników i parametrów przedziału;
    - wyświetlenie wyników (`print()`);
    - wyjście z programu.
7. Architektura programu powinna w maksymalny sposób odgraniczyć część interfejsu użytkownika (wprowadzania/wypisywanie danych) od logiki programu, tj. dane wprowadzaj w oddzielnej klasie a implementacje logiki pozostaw odseparowaną od wprowadzania danych.

Przykładowa sesja:

```
1 – Liniowa  2 – Kwadratowa  3 – Wykładnicza  0 – Wyjście
> 2
a: 1
b: -4
c: 3
Początek przedziału: 0
Koniec przedziału: 4
Krok: 1
--- f(x) = 1.00x^2 - 4.00x + 3.00
Delta: 4.00, wierzchołek: (2.00, -1.00)
Miejsca zerowe: 1.00, 3.00
--- Przebieg na [0.00, 4.00], krok 1.00
x:    0.00   1.00   2.00   3.00   4.00
f(x): 3.00   0.00  -1.00   0.00   3.00
Min: -1.00, max: 3.00
Pole (metoda trapezów): 2.00
```

Wskazówki

1. ALT+INSERT otwiera menu generowania kodu w IntelliJ IDEA.
2. Rzucanie wyjątku:
   ```
   if (a == 0) {
     throw new IllegalArgumentException("Współczynnik a nie może być równy 0");
   }
   ```
3. Przydatne metody: Math.sqrt(), Math.pow(), Math.log().
4. Porównywanie liczb *double* z zerem: używaj tolerancji, np. Math.abs(delta) < 1e-9.
5. Dziedziczenie i implementacja interfejsu:
   ```
   public class MyExtendedClass extends MyBaseClass implements MyInterface {
        // implementacja
   }
   ```
6. Pobieranie danych z klawiatury:
   ```
   Scanner scanner = new Scanner(System.in);
   int myInt = scanner.nextInt();
   double myDouble = scanner.nextDouble();
   String myString = scanner.nextLine();
   ```
7. Wypisywanie na ekran, formatowanie:
   ```
   System.out.println("Hello World"); // zakończone znakiem nowej linii
   System.out.print("Hello World");
   System.out.println(String.format(Locale.US, "%s %d %.2f", myString, myInt, myDouble));
   ```

Teoria

1. JDK, JRE, JVM - co oznaczają te skróty?
2. Czym się różni program napisany w C++ od programu w JAVA pod względem kompilacji (kompilacja do kodu bajtowego vs kompilacja do kodu maszynowego, zalety i wady);
3. Obiektowość w języku JAVA – pakiety, budowa klasy, definicja konstruktora, metody klas, dostępność składowych, metoda main, przeładowanie nazw metod, domyślne argumenty metod.
4. Typy prymitywne - ich ogólna charakterystyka, w tym precyzja `double` i porównywanie liczb zmiennoprzecinkowych.
5. Interfejs (jako konstrukcja programistyczna).
6. Klasa abstrakcyjna.
7. Konwencja nazewnicza klas, zmiennych i metod. Na zajęciach posługujemy się konwencją zgodną z GJS: https://google.github.io/styleguide/javaguide.html

Teoria OBOWIĄZUJE we wszystkich programach (również na kolejnych zajęciach).

Po uzyskaniu zaliczenia na zajęciach, prześlij źródła w archiwum zgodnie z konwencją nazewniczą.